

Proyecto Final

Plataforma inteligente para evaluación y optimización de SQL

Introducción

Construir una plataforma backend que permita evaluar automáticamente consultas SQL enviadas por estudiantes, similar a un juez online, pero orientado al aprendizaje de bases de datos, análisis de rendimiento y optimización de consultas.

El sistema debe permitir que un profesor cree retos SQL, defina esquemas de base de datos, cargue o genere datos de prueba, configure resultados esperados y evalúe automáticamente las soluciones enviadas por los estudiantes.

Además de validar si una consulta produce el resultado correcto, la plataforma deberá analizar su rendimiento y generar recomendaciones en lenguaje natural mediante un asistente inteligente. Este asistente deberá sugerir mejoras como creación de índices, ajustes en la estructura de la consulta, identificación de malas prácticas y posibles versiones optimizadas del SQL enviado.

Objetivo general

Diseñar e implementar el MVP de una plataforma backend para evaluar automáticamente consultas SQL, medir su rendimiento y generar recomendaciones inteligentes de optimización, siguiendo los principios de Clean Architecture, procesamiento asíncrono, ejecución controlada con Docker y organización académica por cursos y evaluaciones.

Reglas del proyecto

Modalidad: trabajo en equipos de máximo 4 estudiantes.

Duración: 5 semanas.

Entregas: 2 entregas parciales.

Arquitectura requerida: Clean Architecture.

Stack sugerido: Node.js + NestJS, PostgreSQL, Redis, Docker Compose, JWT.

Módulos a desarrollar

Módulo 1 — Gestión de usuarios, roles y cursos

Este módulo permite administrar los usuarios de la plataforma y organizar los retos dentro de cursos académicos.

Roles del sistema

La plataforma debe manejar como mínimo los siguientes roles:

- ADMIN
- PROFESSOR
- STUDENT

Permisos esperados

ADMIN

Puede:

- Gestionar usuarios.
- Gestionar profesores.
- Gestionar cursos.
- Consultar información general de la plataforma.

PROFESSOR

Puede:

- Crear cursos.
- Inscribir estudiantes.
- Crear retos SQL.
- Crear evaluaciones.
- Revisar resultados.
- Consultar reportes.

STUDENT

Puede:

- Ver los cursos en los que está inscrito.
- Consultar retos publicados.

- Enviar soluciones SQL.
- Consultar resultados.
- Revisar recomendaciones de optimización generadas por el sistema.

Autenticación

El sistema debe implementar autenticación con JWT.

Cuando un usuario inicia sesión con correo y contraseña, el backend debe entregar un token JWT que permita identificar quién es el usuario y qué rol tiene.

Cada petición protegida debe validar el token y verificar si el usuario tiene permisos para realizar la acción solicitada.

Gestión de cursos

Un curso representa una asignatura o grupo académico.

Ejemplo:

```
{  
  "name": "Bases de Datos II",  
  "code": "BD2-2026",  
  "period": "2026-1",  
  "group": "1",  
  "professorId": "prof-101"  
}
```

Cada curso debe tener:

- Nombre.
- Código o NRC.
- Periodo académico.
- Profesor responsable.
- Estudiantes inscritos.
- Retos asociados.
- Evaluaciones asociadas.

Módulo 2 — Gestión de retos SQL y datos de prueba

Este módulo permite que el profesor cree los retos SQL que serán resueltos por los estudiantes.

Un reto SQL es un problema donde el estudiante debe escribir una consulta o script SQL que produzca un resultado esperado.

Ejemplo de reto

```
{
  "title": "Clientes con más de tres compras",
  "description": "Construya una consulta SQL que retorne los clientes que han realizado más de tres compras.",
  "difficulty": "Medium",
  "tags": ["SELECT", "JOIN", "GROUP BY", "HAVING"],
  "databaseEngine": "PostgreSQL",
  "timeLimit": 2000,
  "status": "published"
}
```

Campos mínimos del reto

Cada reto debe tener como mínimo:

- title
- description
- difficulty
- tags
- databaseEngine
- timeLimit
- status
- courseId
- createdBy

Estados del reto

Estado	Significado
draft	El reto está en construcción
published	El reto está disponible para los estudiantes
archived	El reto ya no está disponible

Gestión de esquemas

Cada reto debe tener un esquema de base de datos sobre el cual se evaluarán las soluciones.

El profesor podrá cargar scripts SQL como:

```
CREATE TABLE customers (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  city VARCHAR(80) NOT NULL  
);
```

```
CREATE TABLE orders (  
  id SERIAL PRIMARY KEY,  
  customer_id INT REFERENCES customers(id),  
  total DECIMAL(10,2) NOT NULL,  
  created_at DATE NOT NULL  
);
```

Gestión de datos de prueba

El profesor podrá cargar datos iniciales para el reto.

Ejemplo:

```
INSERT INTO customers (name, city) VALUES  
( 'Ana Pérez', 'Bogotá'),  
( 'Carlos Ruiz', 'Medellín'),  
( 'Laura Gómez', 'Cali');
```

```
INSERT INTO orders (customer_id, total, created_at) VALUES  
(1, 150000, '2026-01-10'),  
(1, 200000, '2026-01-12'),  
(2, 90000, '2026-01-13');
```

Generador de datos aleatorios

La plataforma debe incluir una funcionalidad para generar datos de prueba de forma automática a partir del esquema definido para el reto.

Este generador permitirá crear escenarios más realistas y evaluar las consultas con mayor volumen de información.

El profesor podrá configurar:

- Cantidad de registros por tabla.
- Rangos de fechas.
- Valores mínimos y máximos para campos numéricos.
- Listas de valores posibles para campos tipo texto.
- Porcentaje de valores nulos permitidos.
- Relaciones entre tablas mediante llaves foráneas.
- Casos borde para validar consultas.

Ejemplo de configuración

```
{
  "table": "orders",
  "rows": 10000,
  "fields": {
    "customer_id": {
      "type": "foreign_key",
      "references": "customers.id"
    },
    "total": {
      "type": "decimal",
      "min": 10000,
      "max": 5000000
    },
    "created_at": {
      "type": "date",
      "from": "2026-01-01",
      "to": "2026-12-31"
    },
    "status": {
      "type": "enum",
      "values": ["PENDING", "PAID", "CANCELLED"]
    }
  }
}
```

Ejemplo de uso

El profesor podría solicitar:

Generar 1.000 clientes, 10.000 órdenes y 500 productos para probar consultas de ventas por ciudad y rango de fechas.

La plataforma debe crear los datos respetando las relaciones entre tablas.

Por ejemplo:

- Cada orden debe pertenecer a un cliente existente.
- Cada detalle de orden debe estar asociado a un producto válido.
- Las fechas deben estar dentro del rango configurado.
- Los valores numéricos deben respetar los límites definidos.

Módulo 3 — Envío y evaluación automática de soluciones SQL

Este módulo permite que los estudiantes envíen sus soluciones SQL y que el sistema las evalúe automáticamente.

Envío de soluciones

El estudiante debe poder seleccionar un reto publicado y enviar una consulta SQL.

Ejemplo de submission:

```
{
  "id": "subm-101",
  "studentId": "user-22",
  "challengeId": "challenge-50",
  "engine": "postgresql",
  "query": "SELECT c.name, COUNT(o.id) AS total_orders FROM
customers c JOIN orders o ON c.id = o.customer_id GROUP BY c.name
HAVING COUNT(o.id) > 3;",
  "status": "QUEUED",
  "createdAt": "2026-04-23T10:30:00Z"
}
```

Estados del submission

Estado	Significado
QUEUED	En espera de evaluación
RUNNING	La consulta se está ejecutando
ACCEPTED	La consulta produjo el resultado correcto

WRONG_ANSWER	El resultado no coincide con lo esperado
SYNTAX_ERROR	La consulta tiene error de sintaxis
TIME_LIMIT_EXCEEDED	La consulta superó el tiempo permitido
RUNTIME_ERROR	Falló durante la ejecución
OPTIMIZATION_REQUIRED	Funciona, pero tiene bajo rendimiento

Evaluación automática

Cuando un estudiante envía una consulta, el sistema debe:

1. Registrar el submission.
2. Enviarlo a una cola de procesamiento.
3. Preparar el esquema de base de datos del reto.
4. Cargar los datos de prueba.
5. Ejecutar la consulta del estudiante.
6. Comparar el resultado obtenido con el resultado esperado.
7. Medir el tiempo de ejecución.
8. Calcular el puntaje.
9. Guardar el resultado.
10. Permitir que el estudiante consulte la retroalimentación.

Ejemplo de resultado

```
{
  "submissionId": "subm-101",
  "status": "ACCEPTED",
  "score": 100,
  "executionTimeMs": 120,
  "tests": [
    {
      "caseId": 1,
      "status": "OK",
      "rowsExpected": 5,
      "rowsReturned": 5
    },
    {
      "caseId": 2,
      "status": "OK",
      "rowsExpected": 8,
      "rowsReturned": 8
    }
  ]
}
```



```
]
}
```

Criterios de evaluación sugeridos

Criterio	Descripción
Resultado correcto	60%
Tiempo de ejecución	15%
Uso adecuado de SQL	10%
Claridad de la consulta	5%
Recomendaciones atendidas o mejora posterior	10%

Módulo 4 — Ejecución aislada y procesamiento asíncrono

Este módulo permite que las consultas no se ejecuten directamente desde la petición HTTP, sino mediante una cola de procesamiento y un runner controlado.

Procesamiento asíncrono con Redis

La plataforma debe usar Redis como cola de trabajos.

Flujo esperado:

API → Redis Queue → Worker SQL → Runner Docker → Resultado → Base de datos

Responsabilidades de la API

La API debe:

- Recibir el submission.
- Guardar el estado inicial.
- Enviar el trabajo a la cola.
- Permitir consultar el estado del submission.

Responsabilidades del worker

El worker debe:

- Consumir trabajos desde Redis.
- Preparar el entorno de ejecución.

- Ejecutar el runner SQL.
- Recibir los resultados.
- Guardar la calificación.
- Actualizar el estado del submission.

Runner SQL con Docker

Cada evaluación debe ejecutarse en un ambiente controlado usando Docker.

El runner debe:

- Crear una base de datos temporal.
- Ejecutar el script de esquema.
- Cargar los datos de prueba.
- Ejecutar la consulta del estudiante.
- Medir el tiempo de ejecución.
- Retornar el resultado.
- Destruir el ambiente al finalizar.

Ejemplo de ejecución

```
docker run --rm \
  --memory 512m \
  --cpus 0.5 \
  -e POSTGRES_PASSWORD=test \
  postgres:16
```

Requisitos mínimos

- La evaluación no debe ejecutarse sobre la base de datos principal.
- El ambiente de evaluación debe ser temporal.
- El runner debe aplicar límite de tiempo.
- El runner debe controlar memoria y CPU.
- El contenedor debe eliminarse al finalizar.

Módulo 5 — Asistente inteligente obligatorio para optimización SQL

Este módulo es obligatorio.

La plataforma debe incluir un asistente inteligente que analice la consulta enviada por el estudiante y genere recomendaciones en lenguaje natural para mejorar su rendimiento, claridad y uso de buenas prácticas SQL.

El asistente no reemplaza la evaluación automática. Su función es apoyar el aprendizaje del estudiante mediante retroalimentación técnica.

El asistente debe analizar

- Consulta enviada por el estudiante.
- Esquema de tablas.
- Campos usados en filtros.
- Campos usados en joins.
- Campos usados en agrupaciones.
- Campos usados en ordenamientos.
- Tiempo de ejecución.
- Posibles problemas de rendimiento.
- Posibles oportunidades de mejora.

El asistente debe generar

- Explicación en lenguaje natural.
- Recomendaciones de optimización.
- Sugerencia de índices.
- Advertencias sobre malas prácticas.
- Propuesta de reescritura de la consulta cuando aplique.
- Explicación del posible impacto de la mejora.

Ejemplo de consulta enviada

```
SELECT *  
FROM orders o  
JOIN customers c ON o.customer_id = c.id  
WHERE c.city = 'Bogotá'  
ORDER BY o.created_at DESC;
```

Ejemplo de respuesta del asistente

La consulta puede funcionar, pero tiene oportunidades de mejora.

Recomendaciones:

1. Evita usar SELECT * si no necesitas todas las columnas.
2. Se recomienda revisar un índice sobre customers.city, porque esa columna se usa como filtro.
3. También puede ser útil un índice sobre orders.customer_id para mejorar el JOIN.
4. Si el ordenamiento por fecha es frecuente, revisa un índice sobre orders.created_at.

Índices sugeridos:

```
CREATE INDEX idx_customers_city ON customers(city);  
CREATE INDEX idx_orders_customer_id ON orders(customer_id);  
CREATE INDEX idx_orders_created_at ON orders(created_at);
```

Ejemplo de reescritura sugerida

Consulta original:

```
SELECT name  
FROM customers  
WHERE id IN (  
    SELECT customer_id  
    FROM orders  
    WHERE total > 100000  
);
```

Consulta sugerida:

```
SELECT DISTINCT c.name  
FROM customers c  
JOIN orders o ON c.id = o.customer_id  
WHERE o.total > 100000;
```

Explicación esperada:

La versión con JOIN puede ser más clara y, dependiendo de los índices existentes, más eficiente que una subconsulta con IN. Se usa DISTINCT para evitar clientes repetidos cuando tienen varias órdenes mayores a 100000.

Formas válidas de implementación

El módulo de IA puede implementarse de cualquiera de las siguientes formas:

Opción 1 — Reglas inteligentes propias

El sistema detecta patrones como:

- Uso de SELECT *.
- Ausencia de filtros.
- Uso de funciones dentro del WHERE.
- Joins sin columnas indexadas.
- Ordenamientos costosos.
- Agrupaciones sobre grandes volúmenes de datos.

Opción 2 — Integración con un modelo de IA

El sistema envía al modelo:

- Consulta SQL.
- Esquema de tablas.
- Tiempo de ejecución.
- Resultado de evaluación.
- Información básica del plan de ejecución, si está disponible.

El modelo genera recomendaciones en lenguaje natural.

Opción 3 — Enfoque híbrido

El sistema combina reglas internas con IA generativa.

Las reglas detectan problemas y la IA ayuda a redactar explicaciones comprensibles para el estudiante.

Módulo 6 — Evaluaciones, resultados y reportes

Este módulo permite usar la plataforma en un contexto académico real.

Evaluaciones o parciales

El profesor debe poder crear evaluaciones compuestas por uno o varios retos SQL.

Ejemplo:

Evaluación: Parcial 1 - SQL avanzado

Reto 1: Consultas con JOIN

Reto 2: Agregaciones con GROUP BY

Reto 3: Subconsultas

Reto 4: Optimización con índices

Duración: 90 minutos

Intentos máximos: 3

Fecha: 15 de mayo de 2026

Configuración de evaluaciones

Cada evaluación puede tener:

- Nombre.
- Descripción.
- Fecha de inicio.
- Fecha de cierre.
- Duración.
- Retos asociados.
- Intentos máximos.
- Curso asociado.
- Visibilidad de resultados.

Componentes principales

1. API Backend

La API debe desarrollarse con Nest

2. Base de datos principal

Es obligatorio el uso de Postgres

3. Redis

Debe utilizarse como cola para procesar los submissions, se recomienda utilizar BullMQ.

4. Worker SQL

El worker debe consumir los trabajos desde Redis.

Responsabilidades:

- Leer submissions pendientes.
- Preparar el runner.
- Ejecutar evaluación SQL.
- Calcular resultados.
- Solicitar recomendaciones al asistente inteligente.
- Guardar el resultado final.

5. Runner SQL

El runner ejecuta la consulta del estudiante en un ambiente controlado.

Responsabilidades:

- Crear ambiente temporal.
- Cargar esquema.
- Cargar datos de prueba.
- Ejecutar consulta.
- Medir tiempo.
- Retornar resultado.
- Eliminar ambiente.

6. Servicio de IA o recomendaciones

Este servicio puede usar reglas, IA generativa o un enfoque híbrido.

Responsabilidades:

- Analizar consultas.
- Generar recomendaciones.
- Sugerir índices.
- Explicar mejoras.
- Proponer versiones alternativas de SQL cuando aplique.

Flujo general del sistema

1. El profesor crea un curso.
2. El profesor inscribe estudiantes.
3. El profesor crea un reto SQL.
4. El profesor carga el esquema de base de datos.
5. El profesor carga o genera datos de prueba.
6. El profesor define el resultado esperado.
7. El profesor publica el reto.
8. El estudiante consulta el reto.
9. El estudiante envía una solución SQL.
10. La API registra el submission.
11. La API envía el submission a Redis.
12. El worker toma el trabajo.
13. El runner prepara la base temporal.
14. Se ejecuta la consulta del estudiante.
15. Se compara el resultado obtenido con el esperado.
16. Se mide el tiempo de ejecución.
17. El asistente inteligente genera recomendaciones.
18. Se guarda el resultado final.
19. El estudiante consulta su retroalimentación.
20. El profesor consulta reportes de calificaciones.

Entrega parcial 1 — Semana 2

El equipo debe entregar:

- Diseño de arquitectura.
- Modelo de dominio.
- Diagrama de componentes.
- Autenticación con JWT.
- Gestión básica de usuarios y roles.
- CRUD de cursos.
- CRUD de retos SQL.
- Carga básica de esquemas.
- Generación inicial de datos de prueba.
- Docker Compose con API, PostgreSQL y Redis.
- Worker SQL en modo inicial o stub.
- Documentación inicial de la API.

Entrega final — Semana 5

El equipo debe entregar:

- Evaluador SQL funcional.
- Envío de submissions.
- Procesamiento con Redis.
- Worker SQL funcional.
- Runner SQL con Docker.
- Generador de datos aleatorios.
- Medición de tiempo de ejecución.
- Comparación contra resultado esperado.
- Asistente inteligente obligatorio.
- Recomendaciones de optimización SQL.
- Sugerencia de índices.
- Reescritura sugerida de consultas cuando aplique.
- Evaluaciones o parciales.
- Reportes por estudiante, reto y curso.
- README completo.
- Video demostrativo.
- Evidencia de ejecución con Docker Compose.

Rúbrica de evaluación

Criterio	Porcentaje
Diseño de dominio y Clean Architecture	10%
API REST, autenticación y roles	10%
Gestión de cursos, retos SQL y evaluaciones	15%
Gestión de esquemas y generación de datos aleatorios	10%
Evaluador automático SQL	20%
Runner SQL con Docker y procesamiento con Redis	15%
Asistente inteligente de optimización SQL	10%
Reportes, leaderboard, documentación y video	10%

Total: 100%

Se debe trabajar en un ambiente colaborativo en Github, se evaluará individualmente los commits realizado por cada estudiante.